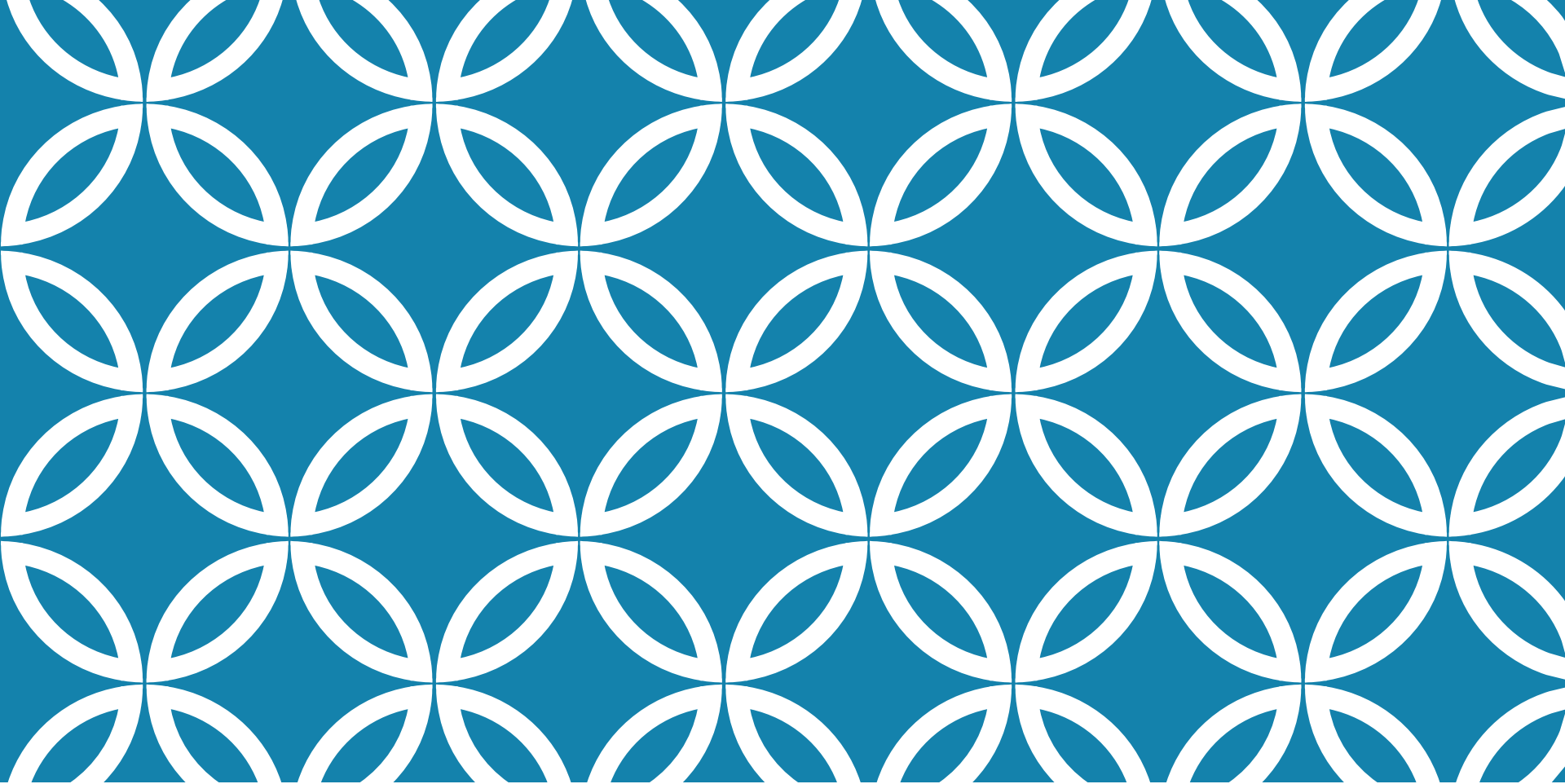


IT SOCIETY

INTRODUCTION

DATABASES

TABLE OF CONTENT

<u>Introduction</u>	5
<u>Relational Databases</u>	6
<u>NoSQL Databases</u>	7
<u>Cloud Databases</u>	8
<u>Columnar Databases</u>	9
<u>Relational Databases</u>	11
<u>MySQL Workbench</u>	12

TOPICS | CREATE

<u>CREATE</u>	15
<u>CREATE DATABASE</u>	16
<u>CREATE TABLE</u>	17
<u>DATA TYPES</u>	18
<u>SHOW</u>	19
<u>SELECT</u>	20

TOPICS | ALTER

<u>ALTER</u>	21
<u>DROP COLUMN</u>	22
<u>ADD COLUMN</u>	23
<u>MODIFY COLUMN</u>	24
<u>CHANGE</u>	26
<u>RENAME</u>	27
<u>ALTER COLUMN</u>	30
<u>AFTER</u>	31
<u>FIRST</u>	32

INTRODUCTION

- **Databases** used to organize and store data.
- **Types** of databases include:
 1. Relational databases
 2. NoSQL databases
 3. Cloud databases
 4. Columnar databases

RELATIONAL DATABASES

- The name comes from the way that data is stored in multiple, related tables.
- Within the **tables**, data is stored in **rows** and **columns**.
- **Structured Query Language (SQL)** is the most common language for reading, creating, updating and deleting data.
- Relational databases work well with **structured data**.
- Examples include **Microsoft SQL Server, Oracle Database, MySQL, PostgreSQL** and **IBM Db2**.

NOSQL DATABASES

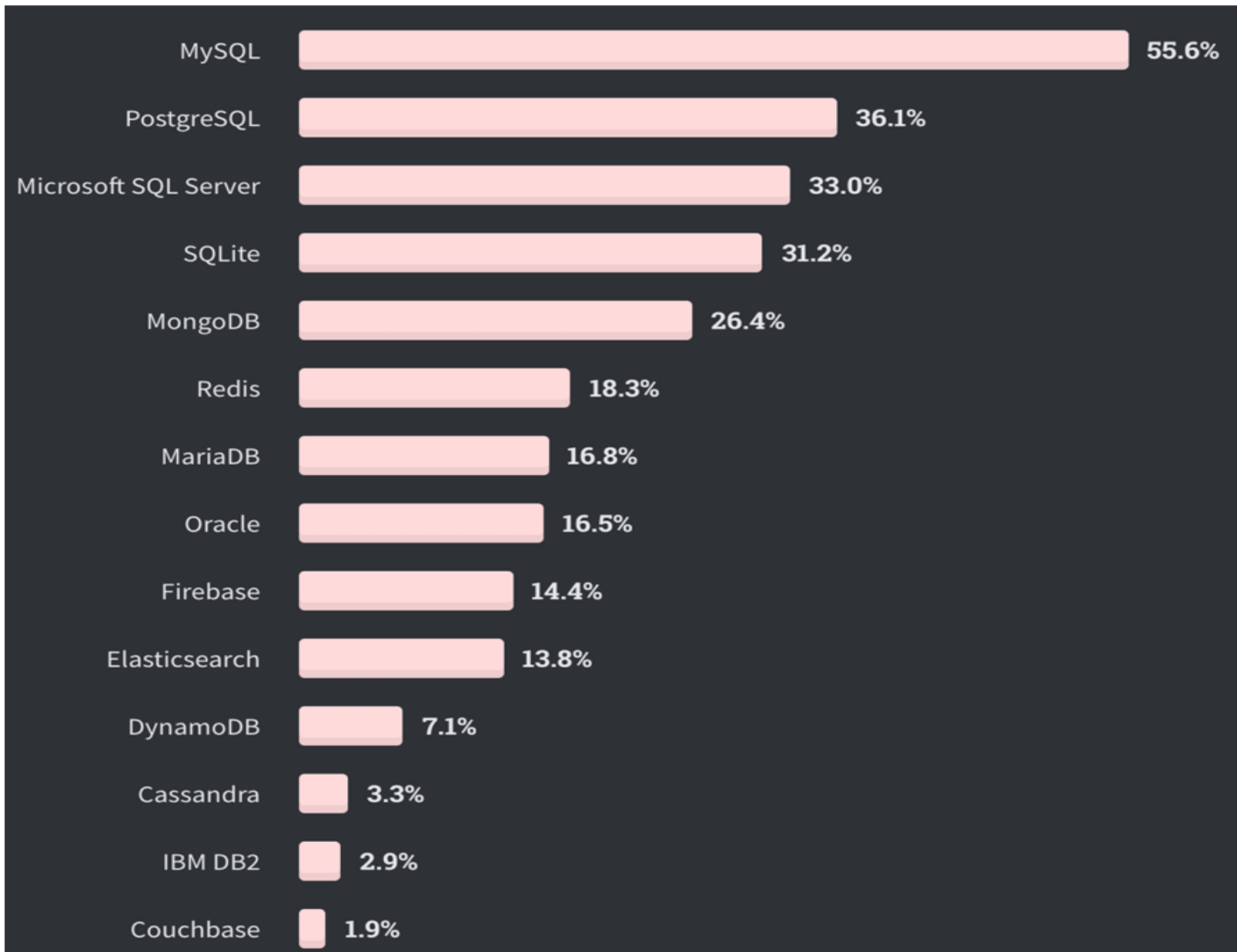
- These types of databases are also sometimes referred to as **non-relational databases**.
- They store **unstructured** or **semi-structured** data.
- One advantage of NoSQL databases is that developers can make changes to the database on the fly, without affecting applications that are using the database.
- Examples include **MongoDB**, and **Couchbase**.

CLOUD DATABASES

- A cloud database refers to any database that's designed to run in the **cloud**.
- Cloud databases offer flexibility and **scalability**, along with **high availability**.
- Cloud databases are also often low-maintenance, since many are offered via a **SaaS** model.
- Examples include **Microsoft Azure SQL Database** and **Oracle Autonomous Database**.

COLUMNAR DATABASES

- Columnar databases **store data in columns** rather than rows.
- Examples include **Cassandra**, and **MariaDB**.



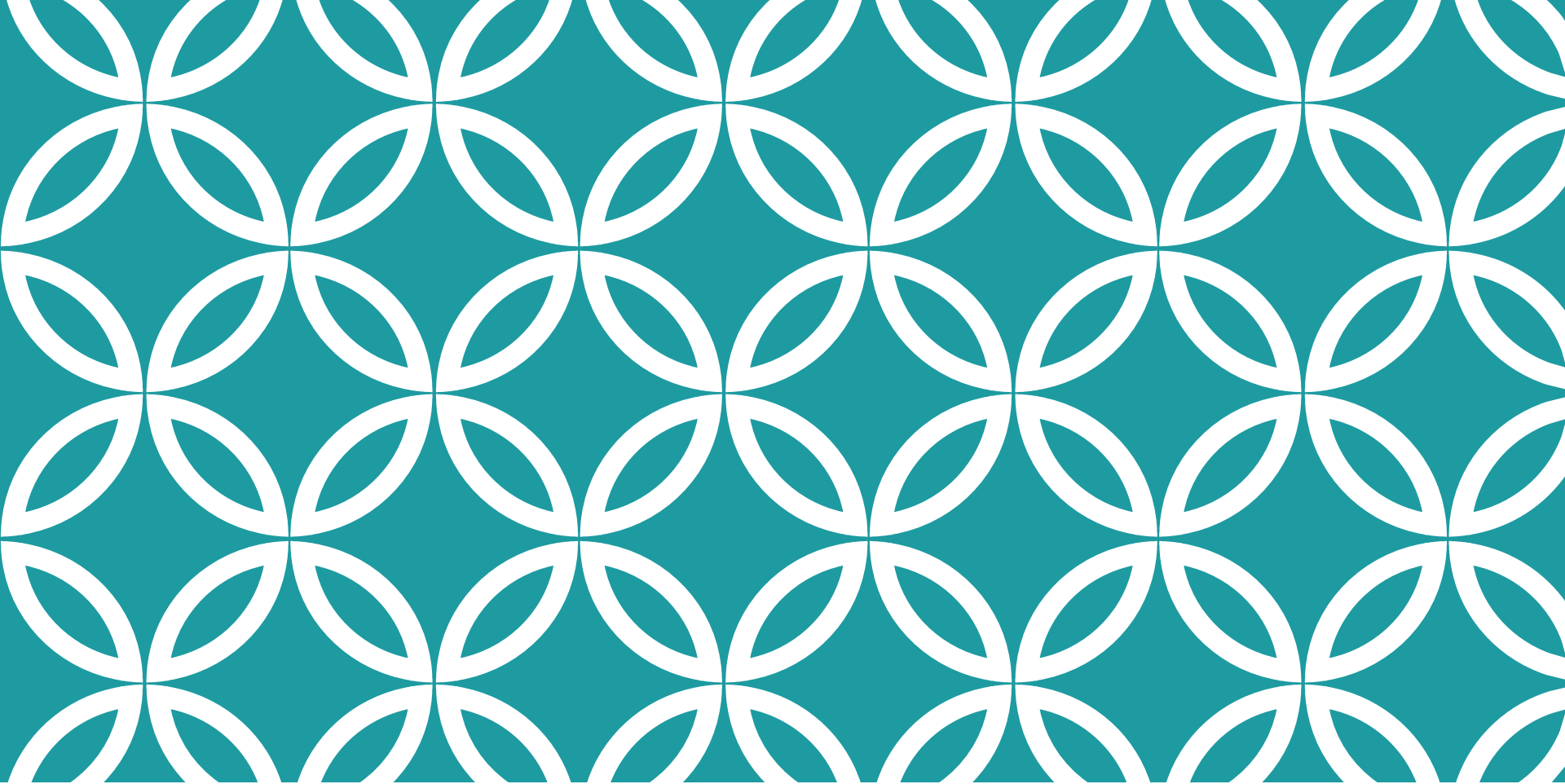


RELATIONAL DATABASES



WORKBENCH

- **MySQL** is an **open source** relational database management system (RDBMS) that's used to store and manage data.
- **MySQL Workbench** provides data modeling, SQL development, and comprehensive administration tools for server configuration, user administration, backup, and much more.



DATA DEFINITION LANGUAGE

CREATE & ALTER

INTRODUCTION

- DDL stands for **Data Definition Language**, which is actually a set of commands used to create a structure and maintain databases.
- The include;
 1. CREATE
 2. ALTER
 3. DROP
 4. TRUNCATE
 5. RENAME

1. CREATE

COMMAND	DESCRIPTION
CREATE DATABASE	Creates a new database
CREATE TABLE	Creates a new table within a database
CREATE USER	Creates a new database account
CREATE VIEW	Creates a virtual table based on a query's result-set
CREATE PROCEDURE	Creates a stored routine that can be called to perform operations
CREATE FUNCTION	Creates a stored routine that returns a single value
CREATE TRIGGER	Creates a database object that automatically executes in response to table events
CREATE EVENT	Creates a scheduled task for the MySQL Event Scheduler

1.1 CREATE DATABASE

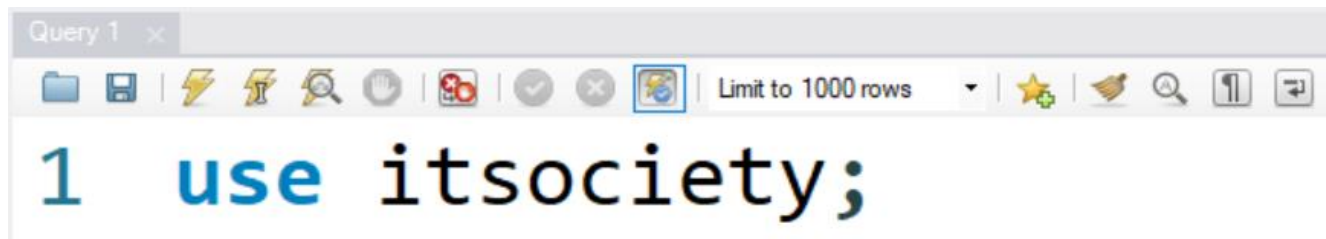
- Let's create a database name itsociety.



A screenshot of a SQL query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and search. The text "Limit to 1000 rows" is visible. The query text is:

```
1 create database itsociety;
```

- Now, we need to select the created db.



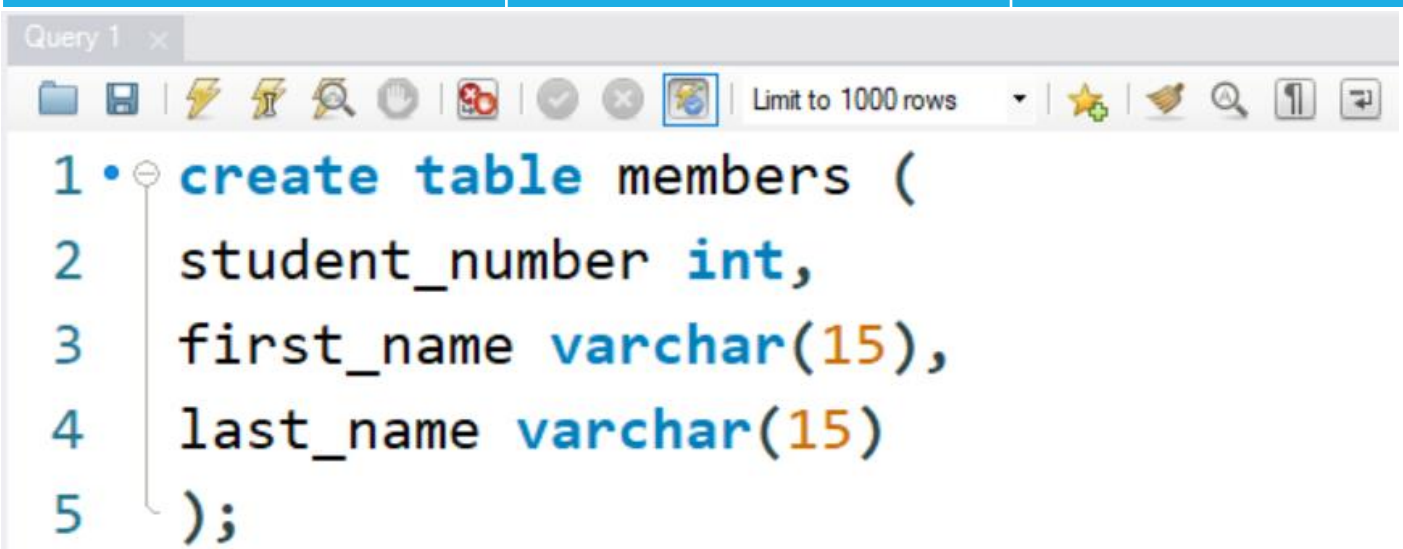
A screenshot of a SQL query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and search. The text "Limit to 1000 rows" is visible. The query text is:

```
1 use itsociety;
```

1.2 CREATE TABLE

- Our table should have the following fields;

student_number	first_name	last_name
----------------	------------	-----------



The screenshot shows a SQL query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL code is as follows:

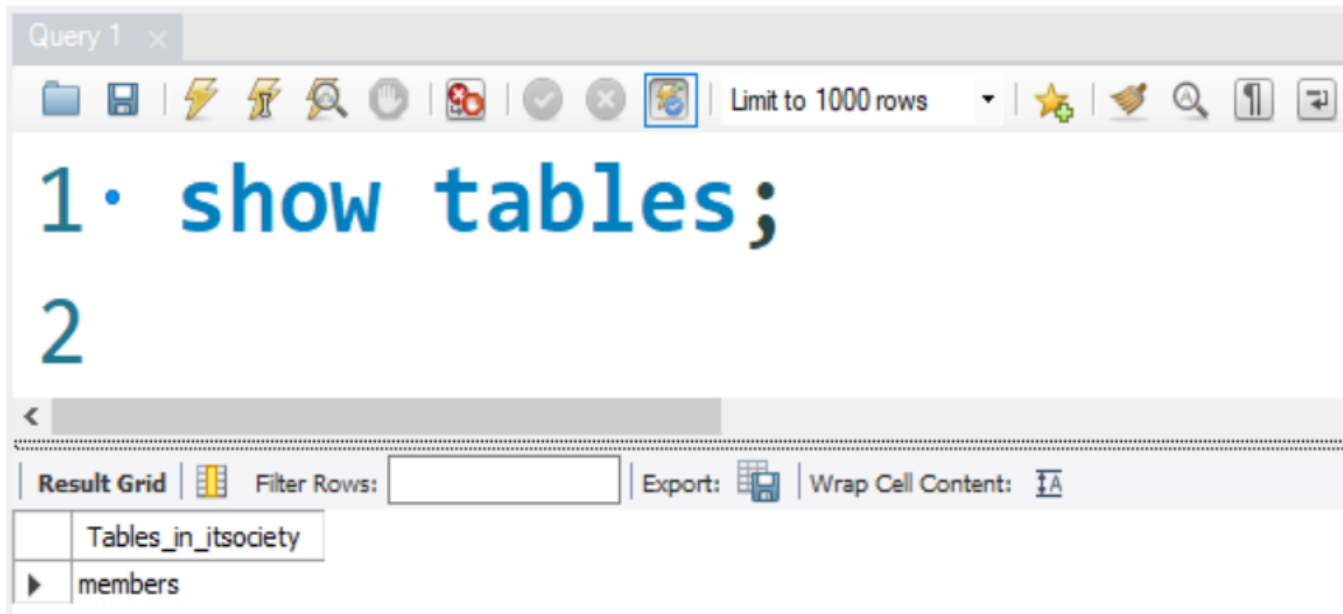
```
1 • create table members (  
2     student_number int,  
3     first_name varchar(15),  
4     last_name varchar(15)  
5 );
```

DATA TYPES

TYPE	EXAMPLE	DESCRIPTION
Numeric	INT	INTEGER data type is a standard numeric type used to store whole numbers.
	DECIMAL(M, D)	Is a fixed-point data type used to store exact numeric values.
String	CHAR(M)	Is used to store fixed-length strings.
	VARCHAR(M)	Is a variable-length string.
Text	TEXT	It stores large amounts of character data.
Date and Time	DATE	Is used to store values that contain a date.
	TIME	Is used to store time-of-day.
	DATETIME	Is used to store values that contain both date and time parts.
	TIMESTAMP	
	YEAR	Is used to store a 4-digit calendar year.

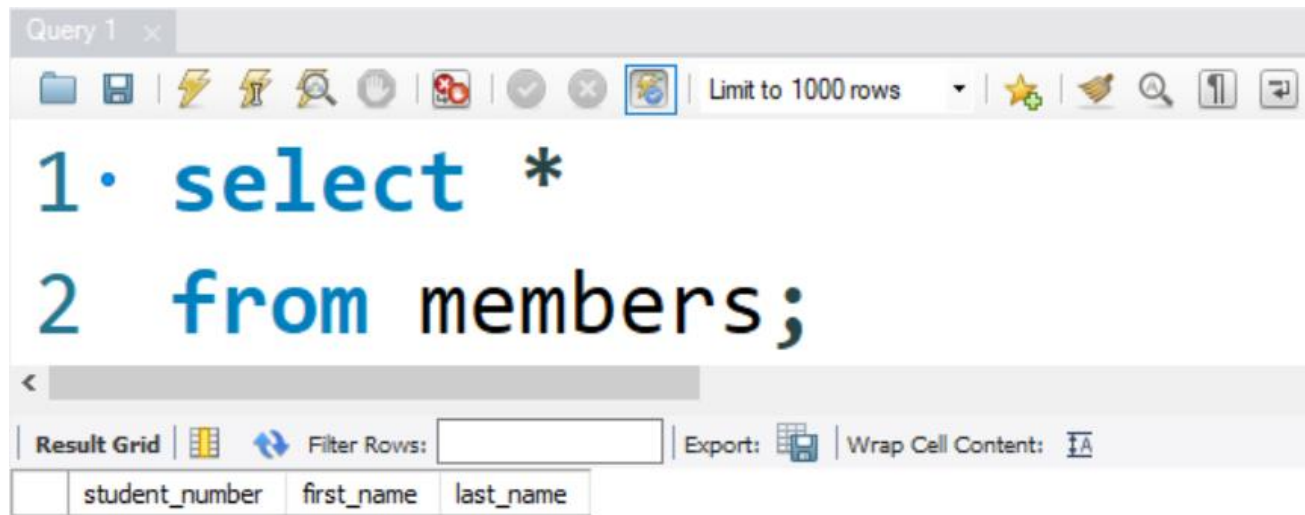
SHOW

- Let's check the available tables.



SELECT

- Let's now view the content of our table.



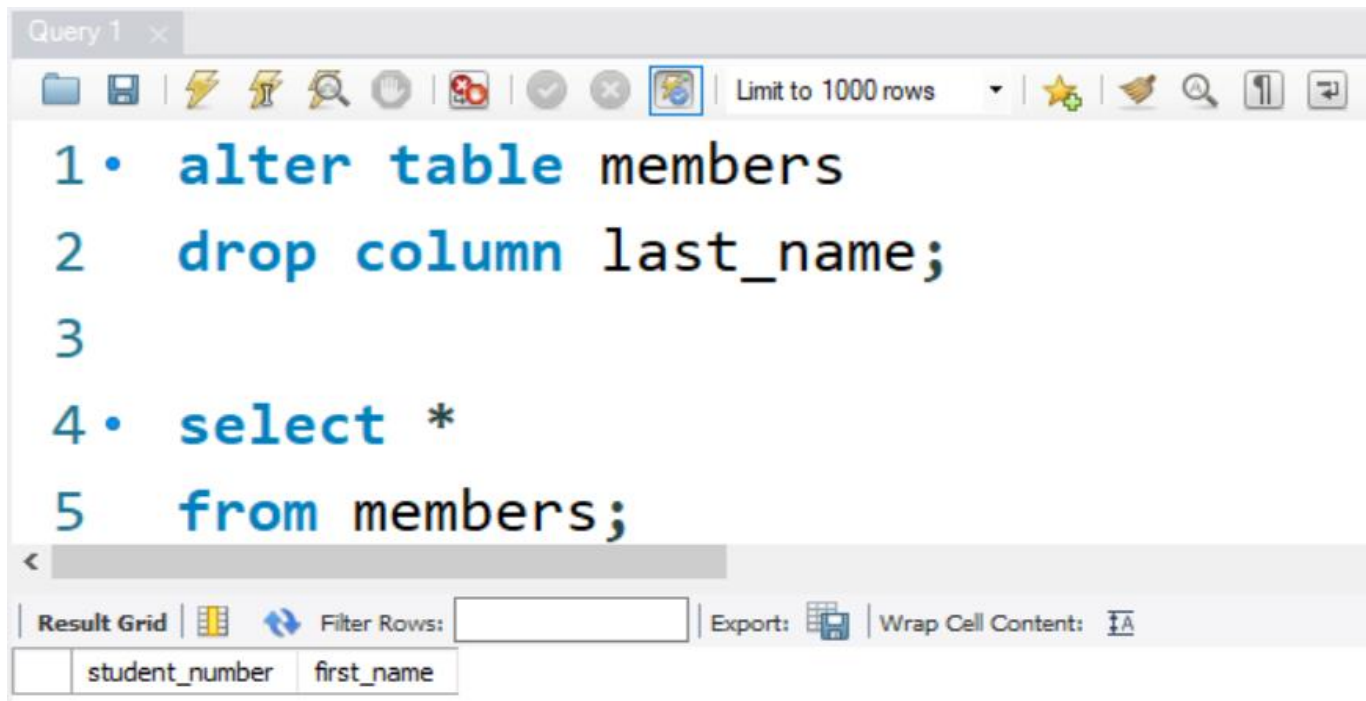
- **NOTE:** * means all columns (everything)

2. ALTER

TARGET	TYPE	DESCRIPTION
DATABASE TABLE	DROP	Remove database.
		Remove column.
	ADD	Add new column/constraint.
	MODIFY	Change data type only (name stays same).
	CHANGE	Rename column AND change data type.
	RENAME	Rename the entire table/column.
	ALTER	Set or drop default value only

2.1 DROP COLUMN

- Suppose we want to get rid of the column last_name, our query can look like this;



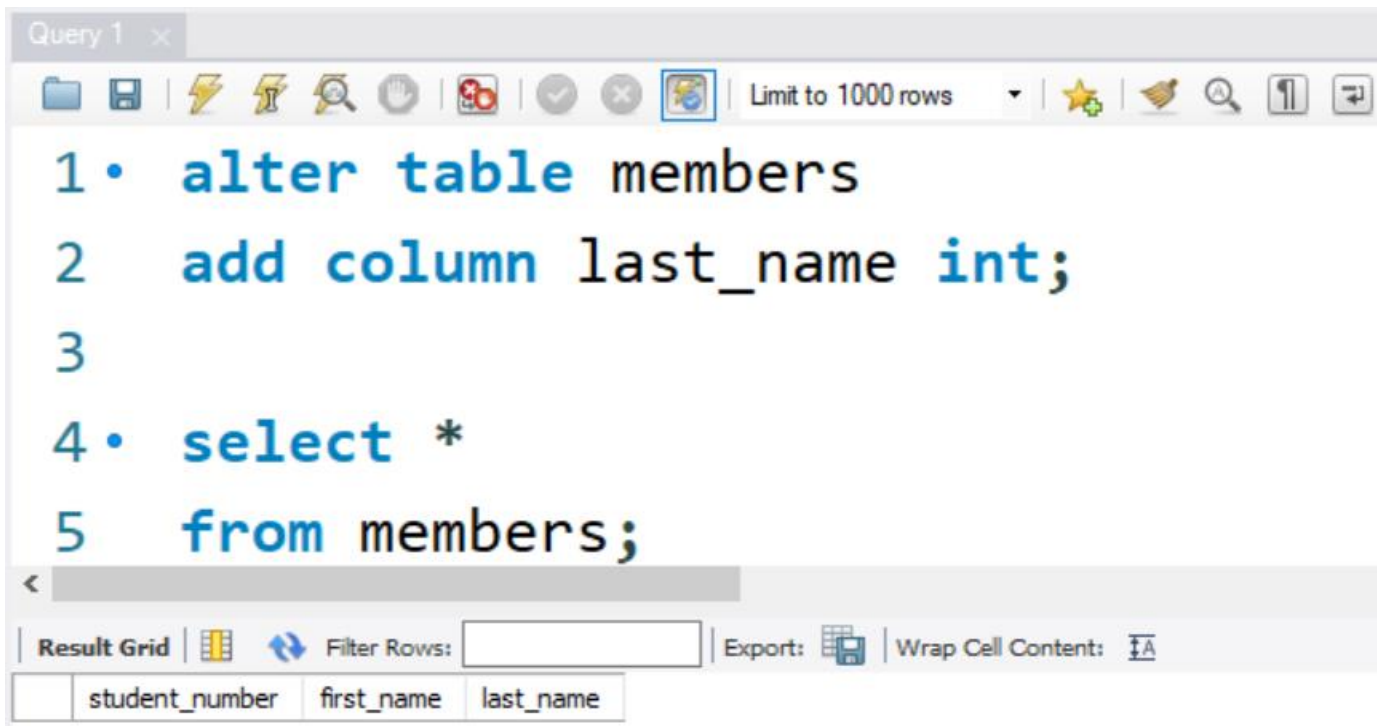
The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and settings, along with a "Limit to 1000 rows" dropdown. The SQL query is as follows:

```
1 • alter table members
2   drop column last_name;
3
4 • select *
5   from members;
```

Below the query editor, there is a "Result Grid" section with a "Filter Rows:" input field and an "Export:" button. The result grid shows two columns: "student_number" and "first_name".

2.2 ADD COLUMN

- Now let's add back the column we just deleted.



The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and settings. The SQL code is as follows:

```
1 • alter table members
2   add column last_name int;
3
4 • select *
5   from members;
```

Below the code editor, there is a "Result Grid" section with a "Filter Rows" input field and an "Export" button. The result grid displays the following columns:

student_number	first_name	last_name
----------------	------------	-----------

2.3 MODIFY COLUMN

- Opppps! We have just used the wrong datatype, let's use the correct one by modifying it.



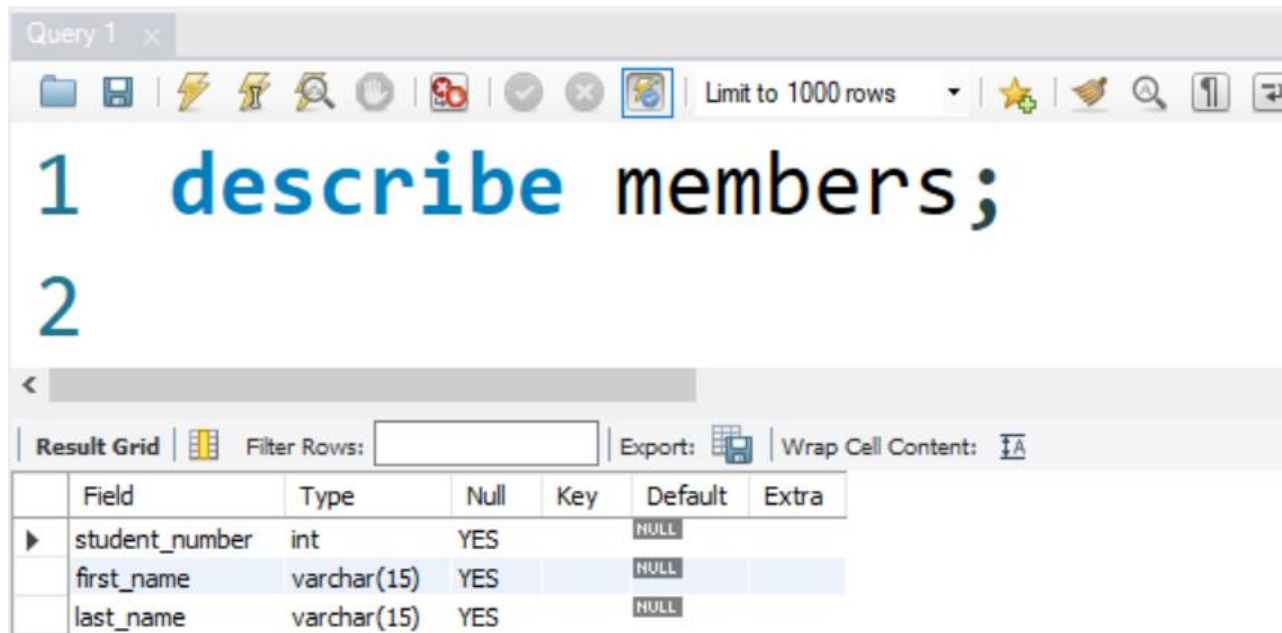
The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL code is as follows:

```
1 • alter table members
2   modify column last_name varchar(15);
3
4 • select *
5   from members;
```

At the bottom, there is a "Result Grid" section with a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. Below this, a table header is visible with columns: student_number, first_name, and last_name.

DESCRIBE

- Let's confirm if we are now using the correct datatype.



The screenshot shows a database query editor window titled "Query 1". The query entered is:

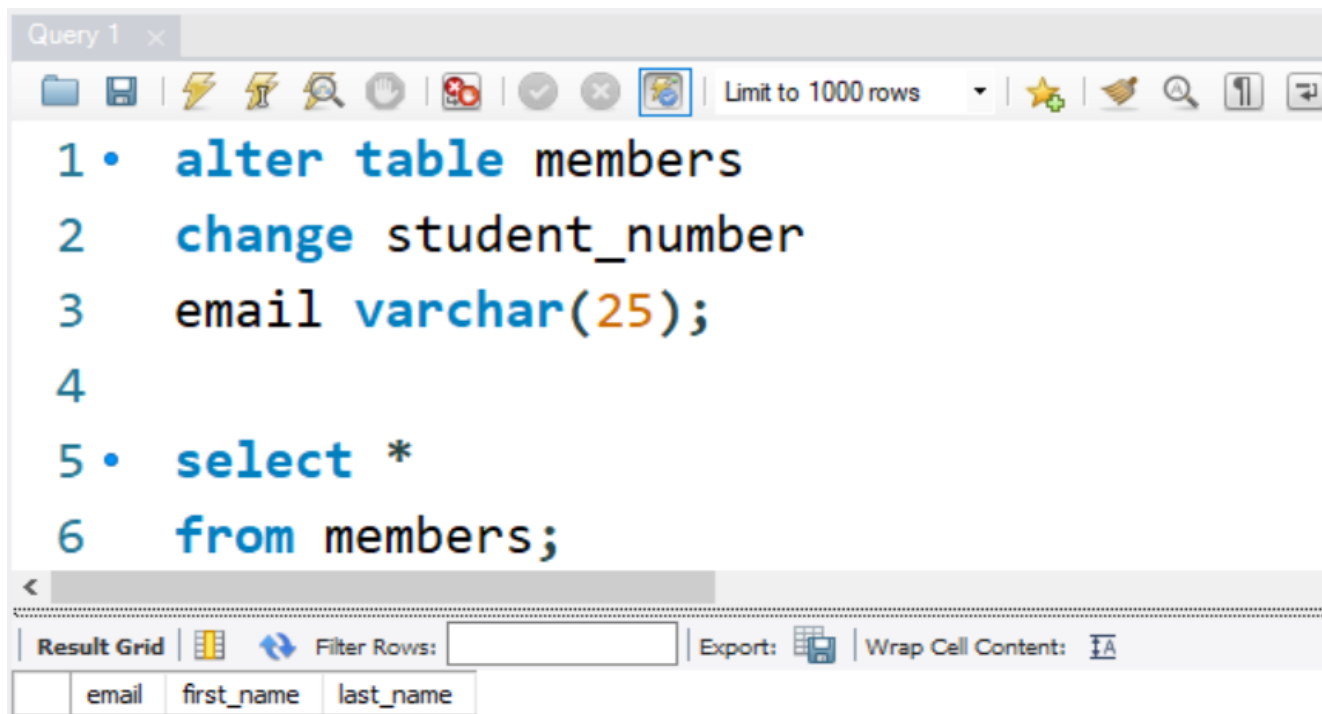
```
1 describe members;  
2
```

Below the query editor, the "Result Grid" is displayed, showing the schema of the 'members' table. The grid has columns for Field, Type, Null, Key, Default, and Extra.

Field	Type	Null	Key	Default	Extra
student_number	int	YES		NULL	
first_name	varchar(15)	YES		NULL	
last_name	varchar(15)	YES		NULL	

2.4 CHANGE

- Suppose we want to make the student_number field change to email with a datatype of varchar();



The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL code is as follows:

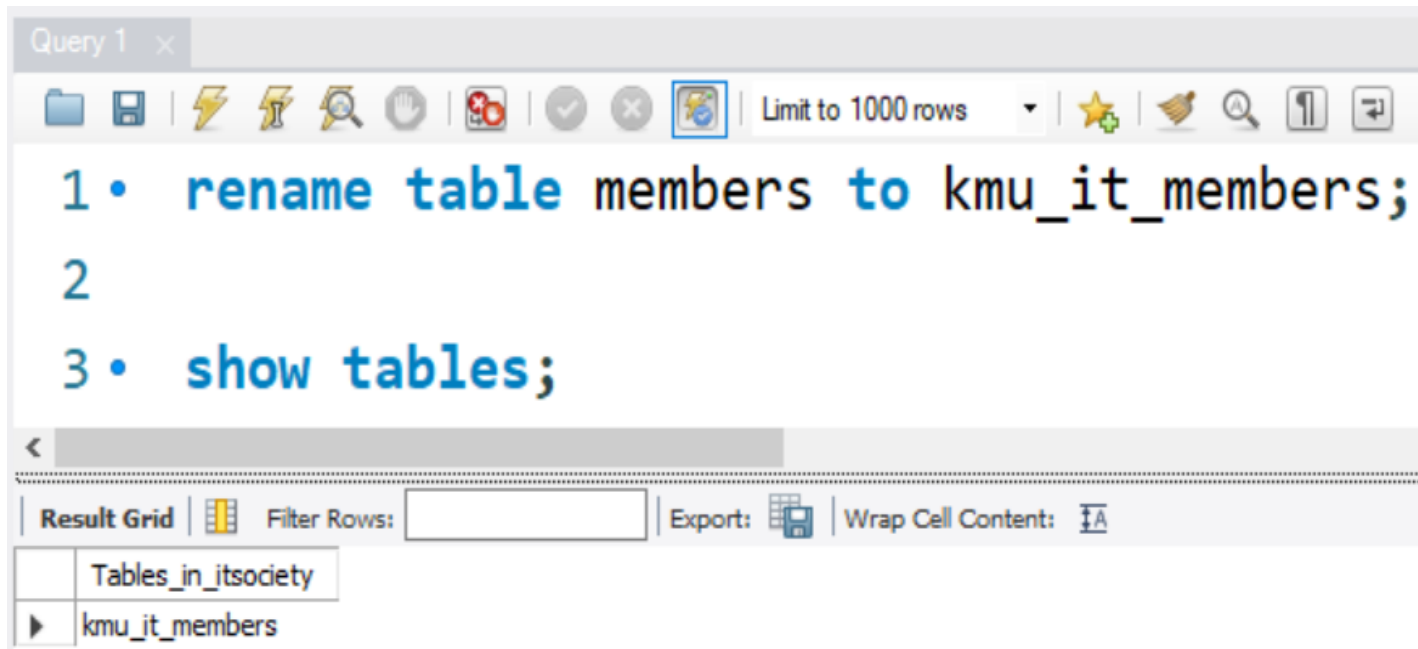
```
1 • alter table members
2   change student_number
3   email varchar(25);
4
5 • select *
6   from members;
```

Below the code editor, there is a "Result Grid" section with a "Filter Rows:" input field and an "Export:" button. The result grid shows the following columns:

email	first_name	last_name
-------	------------	-----------

2.5 RENAME

- **NOTE:** We can rename a table minus using the alter table command as shown below;



The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and settings. The SQL query entered is:

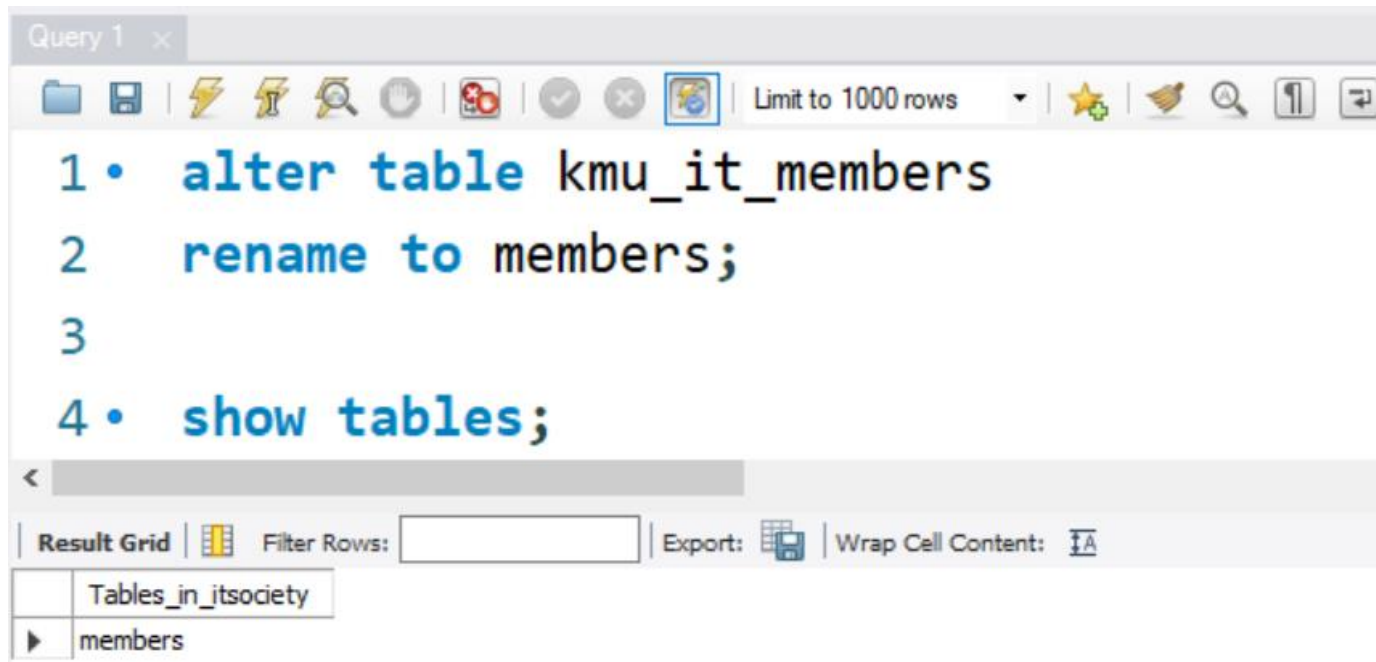
```
1 • rename table members to kmu_it_members;  
2  
3 • show tables;
```

Below the query editor, the "Result Grid" is visible, showing a single row with the table name "kmu_it_members".

Tables_in_itsociety
kmu_it_members

2.5.1 RENAME TABLE

- Okay, let's take rename the table back to its original name but this time around using alter table.



The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL query is as follows:

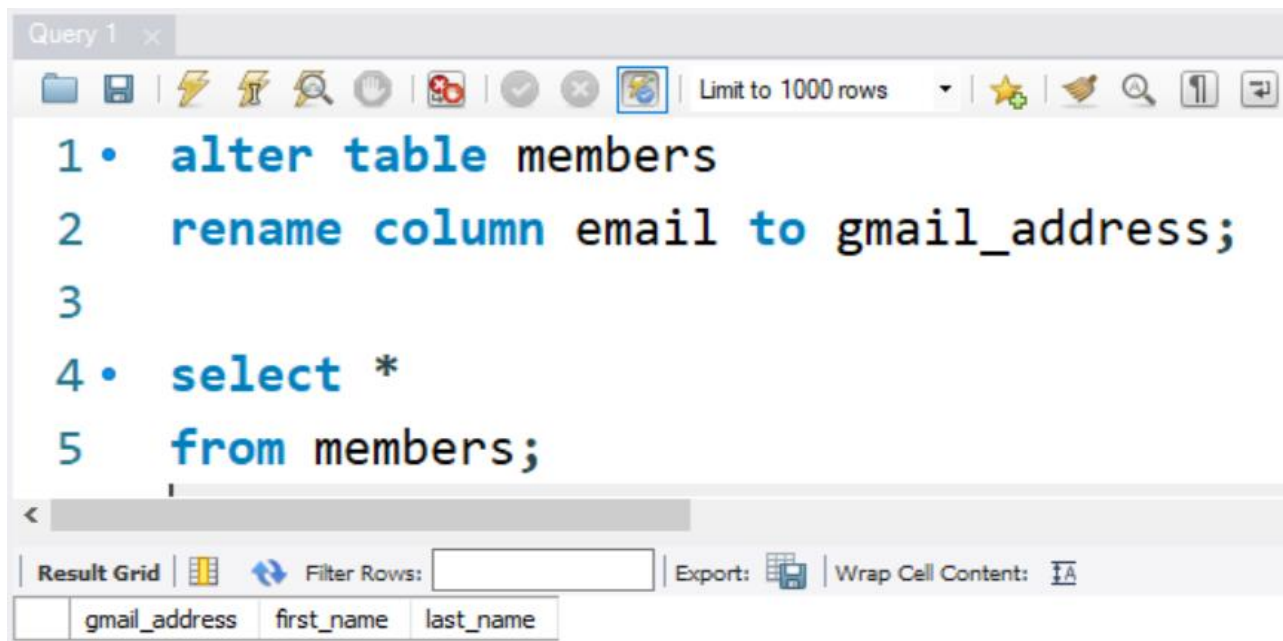
```
1 • alter table kmu_it_members
2   rename to members;
3
4 • show tables;
```

Below the query editor, the "Result Grid" tab is active. It shows a table with two rows:

Tables_in_itsociety
members

2.5.2 RENAME COLUMN

- You've just realized that the emails will be under the Google domain, so you want to make the column to suite the name.



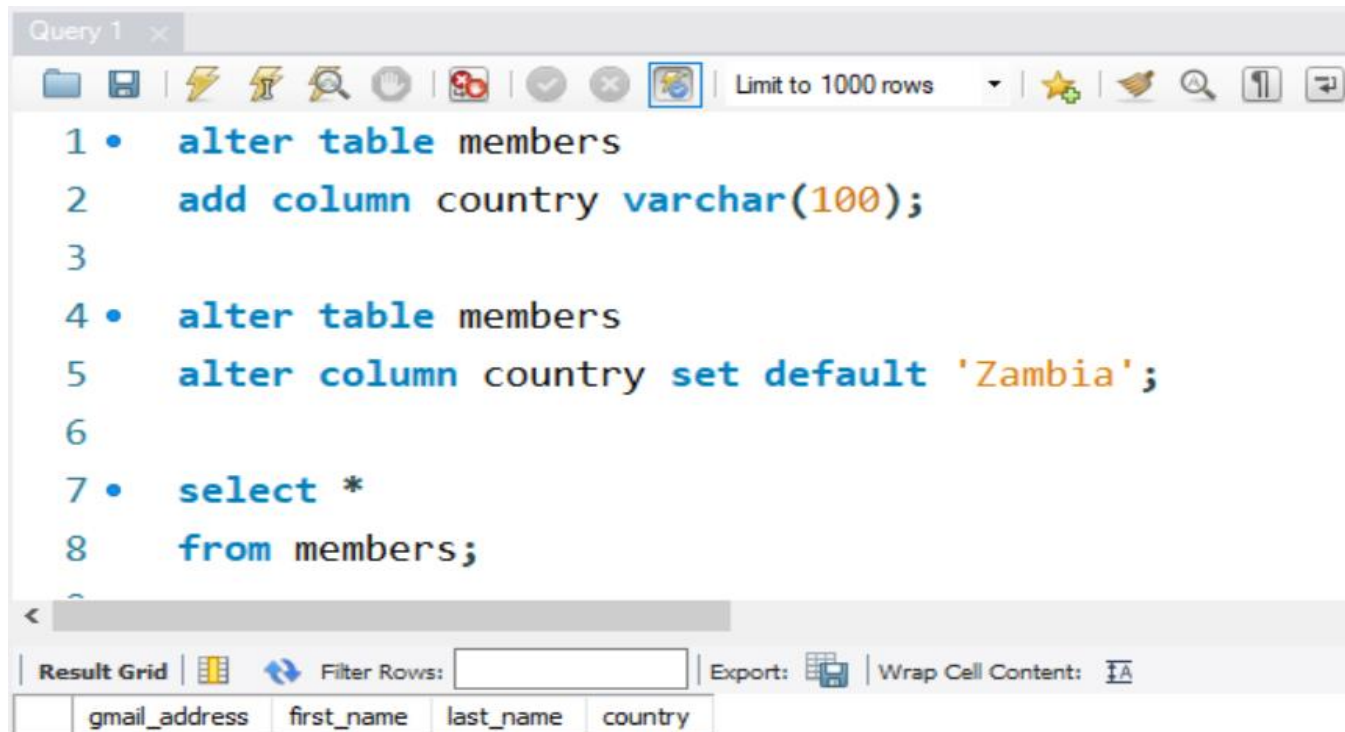
```
Query 1 x
Limit to 1000 rows
1 • alter table members
2   rename column email to gmail_address;
3
4 • select *
5   from members;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

gmail_address	first_name	last_name
---------------	------------	-----------

2.6 ALTER COLUMN

- Now let's add a new column "country" and use the alter command to set Zambia the default country.



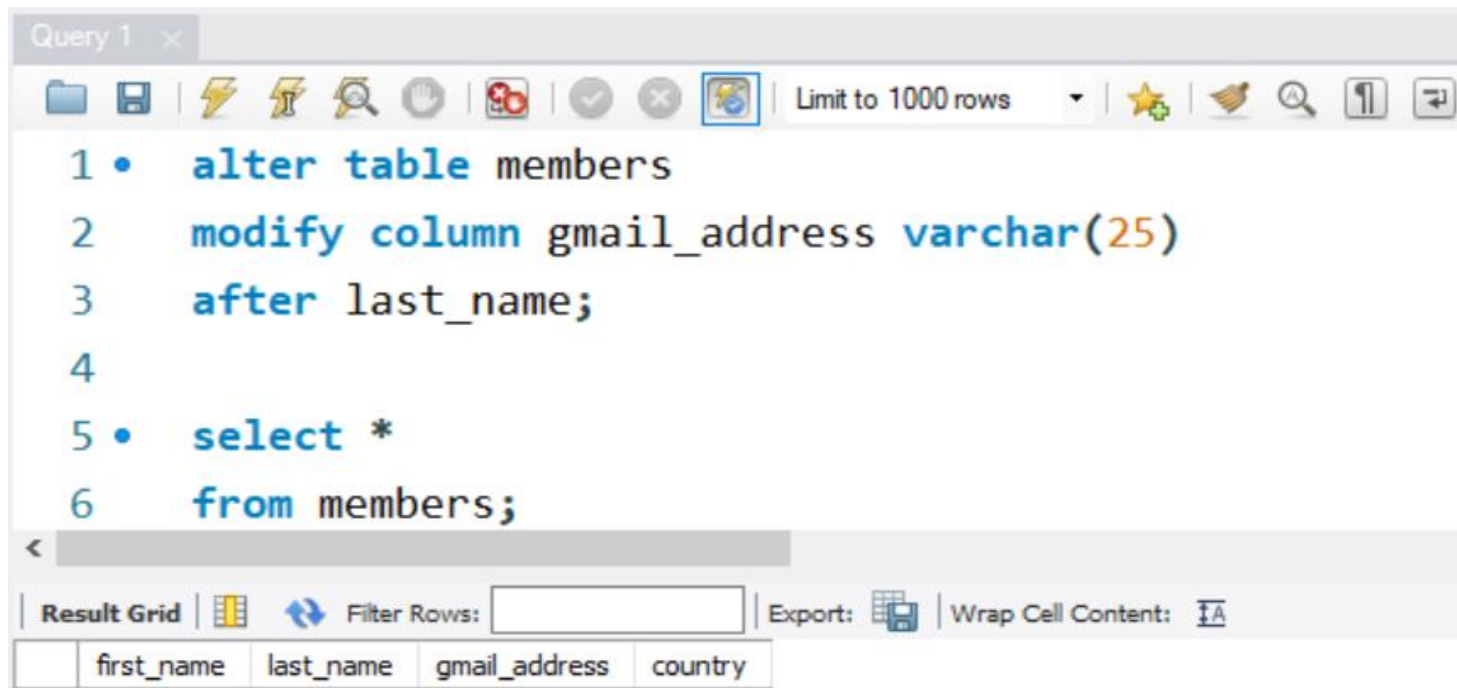
The screenshot shows a database query editor window titled "Query 1". The editor contains the following SQL commands:

```
1 • alter table members
2   add column country varchar(100);
3
4 • alter table members
5   alter column country set default 'Zambia';
6
7 • select *
8   from members;
```

Below the query editor, there is a toolbar with icons for saving, running, and other database operations. The bottom of the window shows a "Result Grid" with columns: gmail_address, first_name, last_name, and country.

AFTER

- You've just realized that gmail_address should be placed after last_name, the command can be;



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, execution, and settings. The main text area contains the following SQL commands:

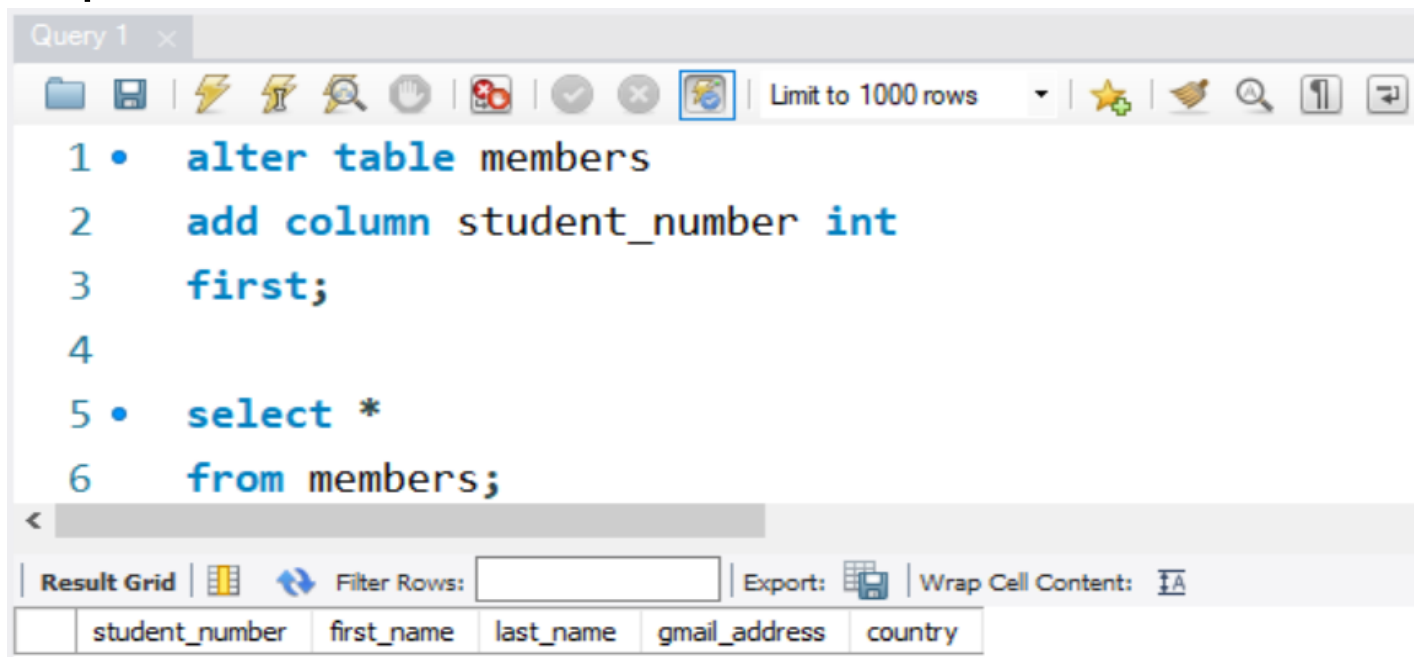
```
1 • alter table members
2   modify column gmail_address varchar(25)
3   after last_name;
4
5 • select *
6   from members;
```

Below the code editor is a horizontal scrollbar. At the bottom, there is a 'Result Grid' section with a 'Filter Rows' input field and an 'Export' button. Below this is a table with four columns: first_name, last_name, gmail_address, and country.

first_name	last_name	gmail_address	country
------------	-----------	---------------	---------

FIRST

- Now, let's bring back the field "student_number" but it must be placed first. **NOTE:** Any new added column, will be placed last without the after or first command.



The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL code is as follows:

```
1 • alter table members
2   add column student_number int
3   first;
4
5 • select *
6   from members;
```

Below the code editor, there is a "Result Grid" section with a "Filter Rows:" input field and an "Export:" button. The result grid displays the following columns:

student_number	first_name	last_name	gmail_address	country
----------------	------------	-----------	---------------	---------

CONCLUSION

- We now have an insight of the **types of databases** we have, for these sessions will be only focusing on **relational databases** because will be dealing with structured data in form of tables.
- Following the statistics of these DBMSs, **MySQL** is the most used, making it ideal for our sessions.
- In this first handout, our focus was just to introduce databases and to understand some uses of the **CREATE** and **ALTER** command, as we progress will continue using these commands by introducing new things.



UP NEXT... DATA MANIPULATION LANGUAGE